অধ্যায় ২২

JavaScript Testing Mastery — Unit Testing, Integration Testing, Mocking, TDD এবং E2E

Hands-on Project: Booking Quality Lab — Testable Booking Workflow

---

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Automated Testing-এর উদ্দেশ্য ব্যাখ্যা করতে পারবে, (২) Node.js Test Runner দিয়ে Unit Test লিখবে, (৩) Mock ও Dependency Injection ব্যবহার করবে, (৪) Integration Test এবং E2E Test-এর পার্থক্য বুঝবে, (৫) TDD-এর Red–Green–Refactor Cycle অনুসরণ করবে, (৬) Test Coverage-এর সীমাবদ্ধতা ব্যাখ্যা করবে এবং (৭) Booking Quality Lab-এর Core Logic যাচাই করবে।

২২.১ কেন Testing জরুরি?

একটি Booking Application-এ Customer Name, Package, Traveler Count, Total Price এবং Status পরিবর্তনের নিয়ম আছে। একটি ছোট পরিবর্তন—যেমন `guests`-এর Limit বদলানো—অন্য Function-এ ভুল ফল দিতে পারে। **Automated Test** হলো এমন Executable Specification যা নির্দিষ্ট Input-এর বিপরীতে প্রত্যাশিত Output বা Behavior যাচাই করে।

Manual Testing প্রয়োজন, কিন্তু একই Scenario বারবার পরীক্ষা করা সময়সাপেক্ষ। Automated Test দ্রুত Regression শনাক্ত করে; তবে Test Pass মানেই Production System সম্পূর্ণ Bug-free নয়। Test কেবল যেসব Behavior পরীক্ষা করেছে, সেগুলোর বিষয়ে Evidence দেয়।

২২.২ Testing-এর স্তর

Test Type	কী যাচাই করে	উদাহরণ	সাধারণ বৈশিষ্ট্য
Unit	ছোট Function বা Module	`calculateTotal()`	দ্রুত, Isolated
Integration	একাধিক Module-এর সহযোগিতা	Service + Repository	Boundary ও Data Flow যাচাই
Component	UI Component	Form Error Message	UI Interaction
E2E	Browser থেকে সম্পূর্ণ Flow	Place → Confirm	বাস্তবের কাছাকাছি, তুলনামূলক ধীর
Regression	আগের Bug ফিরে এসেছে কি না	Cancelled Booking Confirm আটকানো	Bug Fix-এর পরে যোগ হয়

Test Type	কী যাচাই করে	উদাহরণ	সাধারণ বৈশিষ্ট্য
Smoke	Critical Feature কাজ করছে কি না	App Load ও Primary Form	Deploy-এর পরে উপকারী

Test Pyramid একটি সাধারণ নির্দেশনা: দ্রুত Unit Test বেশি, Integration Test প্রয়োজনমতো এবং Critical User Journey-এর E2E Test সীমিত কিন্তু অর্থবহ। এটি কঠোর অনুপাত নয়।

২২.৩ Arrange–Act–Assert Pattern

একটি Test তিনটি ধাপে পরিষ্কারভাবে লেখা যায়: **Arrange** (Input ও Dependencies প্রস্তুত), **Act** (Function চালানো), **Assert** (ফল যাচাই)।

```javascript
import test from 'node:test';
import assert from 'node:assert/strict';
import { calculateTotal } from './booking.mjs';

test('three hill travelers cost BDT 21,600', () => {
  // Arrange
  const packageId = 'hills';
  const guests = 3;
  // Act
  const total = calculateTotal(packageId, guests);
  // Assert
  assert.equal(total, 21600);
});
```

Test Name-এ Behavior উল্লেখ করো। `works fine`-এর চেয়ে `rejects fractional guest count` অনেক বেশি অর্থবহ।

২২.৪ Node.js Built-in Test Runner

Node.js 20+ ইনস্টল থাকলে অতিরিক্ত Framework ছাড়াই `node:test` ও `node:assert/strict` ব্যবহার করা যায়।

```terminal
node --test booking.test.mjs
node --experimental-test-coverage --test booking.test.mjs
```

`test()` একটি Test Case সংজ্ঞায়িত করে। `assert.equal()` Strict Equality পরীক্ষা করে। `assert.deepEqual()` Nested Object/Array-এর Structure তুলনা করে। `assert.throws()` Synchronous Error, আর `assert.rejects()` Rejected Promise যাচাই করে।

```javascript
import assert from 'node:assert/strict';
assert.equal(2 + 3, 5);
assert.deepEqual({ status: 'ok' }, { status: 'ok' });
assert.throws(() => { throw new Error('bad'); }, /bad/);
await assert.rejects(Promise.reject(new Error('offline')), /offline/);
```

২২.৫ Boundary Value এবং Equivalence Partitioning

Input Validation Test-এ কেবল Valid Case যথেষ্ট নয়। ধরো Guests-এর সীমা ১ থেকে ১০। তাহলে ০, ১, ১০, ১১, Fraction, `NaN` এবং Missing Value পরীক্ষা করা দরকার।

```javascript
import { validateBooking } from './booking.mjs';
const base = { customer: 'Amina', packageId: 'beach', guests: 2 };

for (const guests of [0, 11, 1.5, NaN]) {
  assert.throws(() => validateBooking({ ...base, guests }));
}
```

এই Code-এর `assert` Import আগের উদাহরণের মতো থাকতে হবে। Equivalence Partitioning-এ একই আচরণের Input Group থেকে Representative Case বেছে নেওয়া হয়। Boundary Testing-এ Range-এর প্রান্তগুলো গুরুত্ব পায়।

২২.৬ Test Doubles: Stub, Fake, Spy ও Mock

Stub নির্ধারিত Return Value দেয়। **Fake** সরল কিন্তু কার্যকর Alternative Implementation, যেমন In-memory Repository। **Spy** কোনো Function কতবার ও কী Arguments-এ Call হয়েছে তা রেকর্ড করে। **Mock** নির্দিষ্ট Interaction Expectation যাচাই করে। Framework অনুযায়ী Terminology কিছুটা ভিন্ন হতে পারে।

```javascript
const calls = [];
const notifier = {
  async send(type, booking) {
    calls.push({ type, id: booking.id });
  }
};
```

এখানে `calls` Array ব্যবহার করে Notification Interaction পরীক্ষা করা যায়। External Email Service না চালিয়েও Service-এর Behavior যাচাই করা সম্ভব।

২২.৭ Dependency Injection এবং Testability

Hard-coded Dependency থাকলে Test চালানোর সময় Real Database, Email বা Network প্রয়োজন হতে পারে। Dependency Injection ব্যবহার করলে Test-এর সময় Fake Implementation দেওয়া যায়।

```javascript
const service = createBookingService({
  repository: createMemoryRepository(),
  notifier: { async send() {} },
  idFactory: () => 'BK-TEST-1'
});
```

এখানে `createBookingService` এবং `createMemoryRepository` Project-এর `booking.mjs` থেকে Import করতে হবে। Deterministic ID Factory Test Result স্থিতিশীল রাখে।

২২.৮ Integration Testing: একাধিক Module একসঙ্গে

Unit Test Function-এর Contract যাচাই করে; Integration Test পরীক্ষা করে Module-গুলো একসঙ্গে সঠিকভাবে কাজ করছে কি না।

```javascript
const repository = createMemoryRepository();
const events = [];
const service = createBookingService({
  repository,
  notifier: { async send(type) { events.push(type); } },
  idFactory: () => 'BK-001'
});

const booking = await service.place({
  customer: 'Amina', packageId: 'beach', guests: 2
});
await service.confirm(booking.id);
assert.deepEqual(events, ['created', 'confirmed']);
assert.equal((await repository.find(booking.id)).status, 'confirmed');
```

এখানে Service → Repository → Notifier Flow যাচাই হচ্ছে। তবে Real Database-এর Transaction, Network Retry ও Authentication এই Test-এর আওতায় নেই।

২২.৯ Async Tests, Failure Paths ও Cleanup

Async Function Test করার সময় `await` না দিলে Test আগেই শেষ হয়ে যেতে পারে। Error Expect করলে `assert.rejects()` ব্যবহার করো। Test Isolation নিশ্চিত করতে প্রতিটি Test-এ নতুন Repository বানানো ভালো। Shared Mutable State Flaky Test-এর বড় কারণ।

```javascript
await assert.rejects(
  service.confirm('unknown-id'),
  { code: 'NOT_FOUND' }
);
```

Timeout, Fake Timers, Network Stubbing ও Cleanup প্রয়োজন হতে পারে। Fake Timer ব্যবহার করলে `setTimeout`-নির্ভর Test বাস্তব সময় অপেক্ষা না করেও চালানো যায়।

২২.১০ TDD: Red → Green → Refactor

Red: আগে এমন Test লেখো যা নতুন Requirement না থাকায় Fail করে। **Green:** সবচেয়ে সরল Implementation দিয়ে Test Pass করাও। **Refactor:** Behavior না বদলে Code পরিষ্কার করো এবং Test আবার চালাও।

ধরা যাক, Business Rule: Cancelled Booking পুনরায় Confirm করা যাবে না।

```javascript
// Red: আগে Test
assert.throws(
  () => transition({ status: 'cancelled' }, 'confirmed'),
  { code: 'STATUS' }
);
```

এরপর `transition()` Function-এ Allowed Transition Map যোগ করা হয়। Test Pass হলে Refactor করা যায়। TDD-এর উদ্দেশ্য শুধু Test Coverage নয়; পরিষ্কার Interface ও Design Feedback পাওয়া।

২২.১১ Test Coverage: কী বোঝায়, কী বোঝায় না

Line, Branch এবং Function Coverage একটি Codebase-এর কোন অংশ Test Execution-এ এসেছে তা দেখায়। কিন্তু ১০০% Coverage থাকলেও Assertion দুর্বল হলে ভুল Behavior ধরা নাও পড়তে পারে।

```terminal
node --experimental-test-coverage --test booking.test.mjs
```

Coverage Report দেখে Untested Branch শনাক্ত করো। **Meaningful Assertion**, Boundary Case এবং Failure Path Coverage-এর সংখ্যার চেয়ে বেশি গুরুত্বপূর্ণ।

২২.১২ Browser E2E Testing-এর ধারণা

Playwright-এর মতো Tool দিয়ে বাস্তব Browser খুলে Form Fill, Click ও Result Verify করা যায়। নিচেরটি **illustrative example**; Companion Source Code-এ Playwright Dependency Install করা হয়নি, তাই এই E2E Test চালানো হয়েছে এমন দাবি করছি না।

```javascript
// Example only: requires @playwright/test and browser installation.
import { test, expect } from '@playwright/test';

test('visitor places a demo booking', async ({ page }) => {
  await page.goto('http://localhost:8000');
  await page.locator('#customer').fill('Amina Rahman');
  await page.locator('#guests').fill('2');
  await page.getByRole('button', { name: 'Place demo booking' }).click();
  await expect(page.locator('#bookings li')).toHaveCount(1);
});
```

E2E Test-এর জন্য Test Server চালু থাকা, Browser Installation এবং Test Data Isolation নিশ্চিত করতে হবে। Screenshot, Trace ও Video Failure Debugging-এ সাহায্য করে।

২২.১৩ CI Pipeline-এ Tests

Pull Request-এ Automatic Test চালালে Regression Merge হওয়ার আগেই ধরা পড়তে পারে। একটি Minimal GitHub Actions Example:

```yaml
name: Node Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: node --test source-code/chapter-22/booking.test.mjs
```

এটি Repository Root থেকে File Path সঠিক থাকলে চলবে। Production CI-তে Lockfile, Dependency Audit, Lint, Build ও E2E Steps যোগ করা উচিত।

২২.১৪ Practical Project — Booking Quality Lab

এই Project-এ তিনটি স্তর আছে: **Domain Validation**, **Application Service**, এবং **Browser Demo**। সব Source File Companion Source Code-এর `source-code/chapter-22/` Folder-এ রয়েছে।

```
source-code/chapter-22/
├── booking.mjs           # Domain + Service + Memory Repository
├── booking.test.mjs      # Automated Unit/Integration Tests
├── app.mjs               # Browser Interaction
├── index.html            # Form and Booking List
├── style.css             # UI Styling
├── package.json          # npm test Scripts
└── RUN.md                # Execution Guide
```

Project Flow

1. User Customer, Package ও Guests দেয়।
2. `validateBooking()` Input Validate করে এবং Catalog থেকে Total হিসাব করে।
3. `createBookingService()` নতুন Booking তৈরি করে Repository-তে Save করে।
4. Notifier একটি Educational Event Log করে।
5. User Confirm করলে `transition()` State Rule প্রয়োগ করে।
6. Automated Tests Validation, State Transition, Repository Isolation ও Service Integration যাচাই করে।

Run Instructions

```
TERMINAL
cd source-code/chapter-22
node --test booking.test.mjs
node --experimental-test-coverage --test booking.test.mjs
python -m http.server 8000
```

শেষ Command-এর পর Browser-এ `http://localhost:8000` খুলবে। Browser Demo-তে In-memory Data আছে, Refresh করলে হারিয়ে যাবে। এটি ইচ্ছাকৃতভাবে ছোট Educational Application।

গুরুত্বপূর্ণ Reliability Caveat

Demo-তে Repository Save হওয়ার পরে Notifier Fail করলে `place()` Reject করে, কিন্তু Booking ইতোমধ্যে Save হয়ে থাকতে পারে। এটি **Partial Failure**-এর বাস্তব উদাহরণ। Production System-এ Transactional Outbox, Retry Policy এবং Idempotency বিবেচনা করতে হবে। এই অধ্যায়ের Test ইচ্ছাকৃতভাবে Failure Visibility যাচাই করে; Atomicity দাবি করে না।

২২.১৫ সাধারণ Testing Mistakes

- শুধু Happy Path পরীক্ষা করা; Invalid Input ও Boundary বাদ দেওয়া।
- Implementation Detail-এ অতিরিক্ত Assert করে Refactoring কঠিন করা।
- Real API/Clock/Randomness-এর ওপর নির্ভর করে Flaky Test তৈরি করা।
- Async Test-এ `await` বাদ দেওয়া।
- Test Case-গুলোর মধ্যে Shared State রেখে Order-dependent Result পাওয়া।
- Coverage Percentage-কে Quality-এর একমাত্র সূচক ভাবা।
- Mock দিয়ে সবকিছু Pass করিয়ে Real Integration Failure উপেক্ষা করা।
- Browser E2E Test না চালিয়েও UI Verified বলা।

২২.১৬ Exercises

১. `calculateTotal('city', 4)` কত হবে? Test লেখো।
২. Blank Customer Reject হওয়ার Test লেখো।
৩. Guests ০ এবং ১১ হলে Error হয় কি না যাচাই করো।
৪. `pending → cancelled` Transition-এর Test লেখো।
৫. `cancelled → confirmed` Reject হওয়ার Test লেখো।
৬. Notifier Spy দিয়ে `created` Event Count যাচাই করো।
৭. Repository থেকে পাওয়া Object পরিবর্তন করলে Stored Object বদলায় কি না পরীক্ষা করো।
৮. `confirm()`-এ Missing ID-এর Error যাচাই করো।
৯. Notifier Fail করলে `place()`-এর Result কী হয়, Test করো।
১০. E2E Test Plan লেখো: Invalid Form → Error → Valid Form → Booking Confirm।

২২.১৭ Sample Solutions

```javascript
// 1
assert.equal(calculateTotal('city', 4), 15600);
// 2
assert.throws(() => validateBooking({customer:' ',packageId:'city',guests:2}),
{code:'CUSTOMER'});
// 3
for (const guests of [0, 11]) {
  assert.throws(() => validateBooking({customer:'Amina',packageId:'city',guests}),
{code:'GUESTS'});
}
// 4
assert.equal(transition({status:'pending'},'cancelled').status,'cancelled');
// 5
assert.throws(() => transition({status:'cancelled'},'confirmed'), {code:'STATUS'});
// 6
const events=[]; const notifier={async send(type){events.push(type)}};
// place() করার পরে assert.deepEqual(events,['created']);
// 7
// Repository-এর find() ফল পরিবর্তন করে আবার find() করলে আগের Value থাকবে।
// 8
// await assert.rejects(service.confirm('missing'), {code:'NOT_FOUND'});
// 9
// await assert.rejects(service.place(validInput), /mail offline/);
// 10
// Browser Test Plan: blank customer rejected; valid submit adds one item;
// clicking Confirm changes status from pending to confirmed.
```

এই Sample Snippet-এ `assert`, `calculateTotal`, `validateBooking`, `transition` ইত্যাদি পূর্বের উদাহরণ অনুসারে Import করতে হবে। সম্পূর্ণ Runnable Tests `booking.test.mjs`-এ দেওয়া আছে।

২২.১৮ Knowledge Check — ১০টি MCQ

১. Unit Test-এর প্রধান লক্ষ্য কী? A) Small unit behavior B) Hosting C) SEO D) DNS

২. `assert.rejects()` কোন ক্ষেত্রে? A) CSS B) Rejected Promise C) HTML Tag D) SQL Index

৩. Fake Repository-এর সুবিধা কী? A) Fast isolated tests B) Real payment C) HTTPS D) CDN

৪. TDD-এর সঠিক ক্রম কোনটি? A) Refactor–Green–Red B) Red–Green–Refactor C) Green–Red–Refactor D) Deploy–Build–Test

৫. Boundary Testing-এ কোন Input গুরুত্বপূর্ণ? A) Limit-এর প্রান্ত B) শুধু মাঝের Value C) শুধু Valid Email D) Random CSS

৬. 100% Line Coverage মানেই কি Bug-free? A) Yes B) No C) Always D) Only Node.js

৭. Integration Test কী যাচাই করে? A) Module collaboration B) Font Weight C) Icon Size D) Page Color

৮. Flaky Test-এর একটি কারণ কী? A) Shared State B) Pure Function C) Fixed Input D) Isolated Test

৯. E2E Test সাধারণত কী ব্যবহার করে? A) Real browser journey B) Database schema only C) Color palette D)

Code comments

১০. Notifier Failure-এর পরে Booking Save হয়ে থাকলে কী বোঝায়? A) Partial failure B) Atomic success C) Browser bug only D) Syntax error

Answers ও ব্যাখ্যা

1. **A** — ছোট Function/Module-এর Behavior যাচাই।
2. **B** — Promise Rejection যাচাই।
3. **A** — External Database ছাড়া দ্রুত Test চালানো যায়।
4. **B** — প্রথমে Failing Test, তারপর Implementation, শেষে Refactor।
5. **A** — Boundary Value-তে Error বেশি ধরা পড়ে।
6. **B** — Coverage Execution দেখায়, Correctness নিশ্চিত করে না।
7. **A** — Module/Service-এর Interaction যাচাই।
8. **A** — Test Order ও Timing-এর ওপর নির্ভরতা তৈরি হয়।
9. **A** — Browser-এ User Journey যাচাই।
10. **A** — Save সফল হলেও পরের Step Fail করেছে; Transaction Design দরকার।

২২.১৯ Chapter Recap ও পরবর্তী অধ্যায়

Automated Testing হলো Software Quality-এর গুরুত্বপূর্ণ Feedback System। Unit Test Business Rule যাচাই করে, Integration Test Module Collaboration পরীক্ষা করে, E2E Test Critical User Journey কভার করে। Mock/Fake/Spy Dependency আলাদা করে Test চালাতে সাহায্য করে। TDD Interface Design উন্নত করতে পারে। পরবর্তী অধ্যায় **JavaScript Performance Optimization, Profiling এবং Web Application Performance**-এ বাস্তব Measurement, Memory ও Rendering Performance নিয়ে কাজ করব।